

BUILD-ARCHITECTURE.md

Version : V1

Statut : Référence officielle de l'architecture technique de Vitala

Type : Build Blueprint

1. Objectif

Ce document définit l'architecture technique officielle de Vitala.

Il décrit :

- l'organisation du projet ;
- les couches applicatives ;
- les responsabilités de chaque module ;
- les dépendances autorisées ;
- les règles de communication entre les composants.

Toute implémentation doit respecter cette architecture.

2. Principes d'architecture

L'architecture repose sur les principes suivants :

- Domain-Driven Design (DDD)
 - Clean Architecture
 - Modularité
 - API First
 - Offline First
 - Security by Design
 - Testability by Design
 - AI-Ready Development
-

3. Architecture générale

Le système est composé de cinq couches principales :

1. Présentation (UI)
2. Application
3. Domaines métier
4. Infrastructure
5. Plateforme

Chaque couche possède une responsabilité unique.

4. Couche Présentation (Presentation Layer)

Responsabilités :

- écrans
- widgets
- navigation
- gestion des états
- formulaires
- interactions utilisateur

Cette couche ne contient aucune logique métier.

Elle communique uniquement avec la couche Application.

5. Couche Application

Responsabilités :

- orchestration des cas d'usage
- validation métier simple
- coordination entre domaines
- appels aux APIs
- gestion des transactions fonctionnelles

Cette couche ne connaît pas les détails techniques de stockage.

6. Couche Domaine

Chaque domaine est autonome.

Domaines officiels :

- UDI
- Flash
- Mission
- Trust

Chaque domaine contient :

- entités
- objets de valeur
- services métier

- règles métier
- interfaces des repositories

Les domaines ne dépendent jamais entre eux de manière circulaire.

7. Couche Infrastructure

Responsabilités :

- implémentation des repositories
- accès Supabase
- stockage des médias
- synchronisation
- notifications
- audit
- services externes

Cette couche implémente les interfaces définies dans les domaines.

8. Couche Plateforme

Composants transverses :

- Authentification
- Base de données
- Storage
- Sync
- Monitoring
- Configuration
- Logging
- Sécurité

Cette couche fournit les services communs à toute l'application.

9. Architecture Frontend

Le frontend est organisé par domaines.

Structure logique :

- Core
- Shared
- Features
- Design System
- Services

- Navigation

Chaque fonctionnalité est indépendante.

10. Architecture Backend

Le backend est organisé autour des APIs officielles.

Chaque domaine possède :

- contrôleurs
- services
- validation
- accès aux données

Les domaines communiquent uniquement via des contrats définis.

11. Architecture des données

La base de données suit le Domain Model officiel.

Chaque domaine possède ses propres tables.

Les domaines d'intelligence consomment les données via des projections ou des lectures dédiées.

Aucun moteur d'intelligence ne modifie directement les tables métier.

12. Architecture Offline First

Le client gère :

- Local Workspace
- Drafts
- Cache
- Outbox

Le serveur gère :

- Sync API
- sync_operation
- sync_checkpoint
- sync_conflict

La synchronisation constitue une couche d'infrastructure et non une logique métier.

13. Architecture des moteurs d'intelligence

Les moteurs suivants sont indépendants :

- Radar
- Veille
- Recommandation
- Search

Ils :

- consomment les données autorisées ;
 - produisent des analyses ou recommandations ;
 - ne modifient jamais directement les domaines métier.
-

14. Dépendances autorisées

Les dépendances suivent uniquement cette direction :

Présentation

→ Application

→ Domaine

→ Infrastructure

→ Plateforme

Les dépendances inverses sont interdites.

Les dépendances circulaires sont interdites.

15. Communication entre domaines

Les domaines communiquent uniquement par :

- APIs officielles ;
- événements métier ;
- projections en lecture.

L'accès direct aux données d'un autre domaine est interdit.

16. Gestion des erreurs

Toutes les erreurs doivent être :

- typées ;

- journalisées ;
- traçables ;
- transformées en réponses API cohérentes.

Les erreurs techniques ne doivent jamais être exposées directement aux utilisateurs.

17. Sécurité

Toutes les communications doivent respecter :

- authentification via Supabase Auth ;
 - Row Level Security (RLS) ;
 - validation des entrées ;
 - contrôle des permissions ;
 - journalisation des actions sensibles.
-

18. Performances

L'architecture doit favoriser :

- modularité ;
 - mise en cache ;
 - pagination ;
 - chargement progressif ;
 - optimisation des requêtes ;
 - évolutivité horizontale.
-

19. Évolutivité

L'ajout d'un nouveau domaine doit être possible sans modifier les domaines existants.

Chaque nouveau domaine devra :

- posséder son propre modèle métier ;
 - exposer ses propres APIs ;
 - gérer ses propres données ;
 - respecter les conventions du Build Blueprint.
-

20. Critères de conformité

Une implémentation est conforme uniquement si :

- toutes les couches sont respectées ;
 - aucun domaine n'est couplé de manière interdite ;
 - toutes les APIs respectent leurs contrats ;
 - la base de données respecte le Domain Model ;
 - les règles du document "AI Development Rules" sont appliquées.
-

21. Conclusion

Cette architecture constitue la référence officielle pour tous les développements de Vitala.

Aucune implémentation ne peut s'en écarter sans mise à jour préalable du Build Blueprint.